DELL × NVIDIA AI HACKATHON · SAN FRANCISCO

WreckLess

Reckless red teaming, safely sandboxed — so your production agents wreck less.



47–80 s
advisory → validated patch

5 / 5
fresh live runs, CVE-2024-21513

0 bytes
of your code to a cloud model

An always-on, local-first security swarm on one Dell GB10. It exploits, patches and proves the fix. You approve the merge.

THE PROBLEM

Nobody has time to prove an alert is real.

Every week a dependency in your stack gets a critical advisory. Someone on the security team has to work out whether **your** code is actually exposed, write a proof, write a fix, and show the fix holds.

Scanners hand over alerts

but nobody has time to prove any one of them is real.

Proving exposure is manual

a proof, a fix and a regression test, by hand, for every advisory.

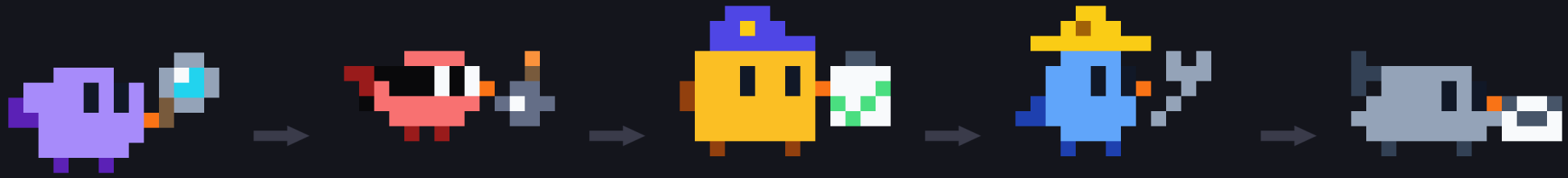
The AI shortcut

usually means sending your source code to a cloud model.

WreckLess does the whole loop on a box under the desk. Every model runs there.

HOW IT WORKS

Five birds, one state machine.



SCOUT

reads the advisory

RALPH

writes & runs an exploit

VERIFIER

re-runs it, fresh nonce

FELIX

patches the root cause

CONDUCTOR

fix card to Slack; a human approves

No proof → rejected, nothing patched

3 checks pass → human clicks Approve

PROOF, NOT ALERTS

No agent grades its own work.



VERIFIER

deterministic code, not a model

- Held-out regression + static checks — written by us, never shown to Felix
- Ralph's exploit, re-run — must now **FAIL**
- The victim app's own functional tests — must still **PASS**

PATCH VALIDATED

The exploit has to be real: the vulnerable code itself must write a random nonce into a file. The Verifier re-runs it on look-alike copies in random order — two vulnerable, one control — and must hit both and miss the control.

LOCAL-FIRST

Every model runs on this box.

DELL PRO MAX · NVIDIA GB10 · 128GB UNIFIED MEMORY

Orchestrator

queue + per-item state machine

vLLM · loopback only

:8000 Scout + Ralph — Qwen3-Coder-30B-A3B

:8001 Felix — Qwen3-Coder-Next 80B-A3B

:8002 victim LLM — Qwen2.5-Coder-7B

OpenShell sandbox

deny-all egress · read-only root — exploit runs here

→ **Slack:** the fix card and Porter's replies; the exploit only when an approver asks (logged)

→ **GitHub:** one draft PR, only after a human clicks Approve

✕ **Cloud LLM APIs:** never. Blocked by airgap_lockdown.sh

ALWAYS-ON

It keeps working between advisories, and you can talk to it.

```
wreckless daemon --slack-listener --porter
```

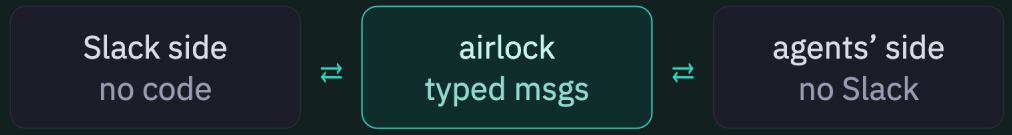
Scout · every 10 min
polls the advisory cache and queues new matches

Ralph · the rest of the time
hunts continuously, skips repeats, yields to queued work

Felix · one item at a time
works the queue; every outcome posts a card

A systemd unit that restarts on failure and survives a reboot.

Porter, the sixth bird: Slack through an airlock



- **No open port.** Outbound WebSocket only; lsof shows no listener.
- **A briefing, not the code.** The model can't print source it never got.
- **change:** re-runs the Verifier and all 3 checks; Approve is bound to the exact commit.

```
@WreckLess why is this dangerous?
```

CASE STUDY · CVE-2024-21513

LangChain runs eval() on database values

CRITICAL `langchain-experimental 0.0.15 - 0.0.20` `fixed in 0.0.21`

Who was targeted

Any app answering questions over a database with LangChain's VectorSQLDatabaseChain — the "chat with your data" pattern. The target is the server running the app. Disclosed, not a named incident; not in CISA's KEV list.

How it was exploited

The chain passes every value that comes back from the database through eval(). An attacker needs no database access — only enough influence over the question to make the returned value Python code.



RALPH

writes & runs the exploit

attacker's question → LLM writes the SQL → DB returns the payload → eval() runs it

CASE STUDY, CONTINUED

Felix matched upstream's fix — without ever seeing it.

5/5

fresh runs · live models · the OpenShell sandbox

47s · 62s · 68s · 71s · 80s

advisory → validated patch

Upstream's fix (commit 7b13292)

```
def _try_eval(x):
    try:
        - return eval(x)
        + return ast.literal_eval(x)
    except Exception:
        return x
```

Felix's fix (local 80B model, rewrites the body)

```
def _try_eval(x):
+   if isinstance(x, str):
+       try:
+           return ast.literal_eval(x)
+       except (ValueError, SyntaxError):
+           return x
+   return x
```

Upstream shipped no test with this fix. Ours is derived from the diff, not the maintainer's test.

MORE CASES

Four more cases, pipeline-verified.

Case	What it is	--no-llm	Hardening probes
CVE-2023-44467	PALChain's denylist misses __import__	3/3 ALL PASS	3/4
CVE-2025-3248	Langflow: unauthenticated /validate/code execs decorators	3/3 ALL PASS	0/3 (auth-only, like upstream)
CVE-2024-23334	aiohttp: follow_symlinks path traversal	3/3 ALL PASS	3/3
WL-2024-RAG-EXFIL	a poisoned message makes a RAG answer leak a private key	3/3 ALL PASS	4/4

Same recipe as CVE-2024-21513: replica, reference exploit and patch, held-out tests. **Live-model runs are next.** Hardening probes attack the patch itself; Langflow’s upstream fix only adds login.

THE STACK

Three models, one unified pool.

:8000 — Scout + Ralph

Qwen3-Coder-30B-A3B-Instruct, AWQ 4-bit

60–73 tok/s · exploit crafting

:8001 — Felix

Qwen3-Coder-Next 80B-A3B, 4-bit

23–33 tok/s · patch quality

:8002 — victim LLM

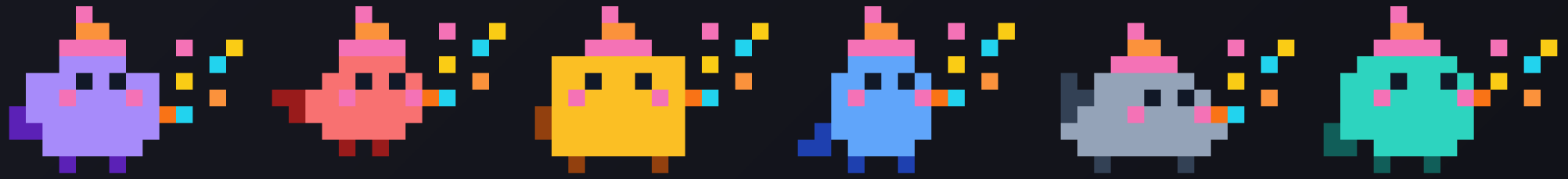
Qwen2.5-Coder-7B-Instruct, AWQ

the assistant Ralph attacks

Dell Pro Max · NVIDIA GB10 · 128GB unified memory · ~63GB weights resident · OpenShell sandbox

WreckLess

Reckless red teaming, safely sandboxed — so your production agents wreck less.



47–80 s · 5/5 live runs · 0 bytes of code to the cloud

github.com/djang-c/WreckLess-Cyber-Security

Demo video: [link] · Team: [names]

Built in a weekend on one desk-side box.